Technical Report:

Character-Level Shahnameh Language Modeling with a Two-Layer Deep LSTM

Code-level analysis of the local Google Colab implementation

Source analyzed: `ca4_shahnameh_deep_lstm_local_colab.py`

Abstract

This report provides a code-faithful technical analysis of a Google Colab-oriented Python implementation for character-level Shahnameh language modeling. The active pipeline reads a user-provided `shahnameh.txt` file, normalizes Persian text, creates a deterministic record-level training/validation split, builds a character vocabulary, and forms non-overlapping next-character prediction sequences. The model uses two 512-unit LSTM layers with a 256-dimensional embedding, dropout, a GELU projection, and a softmax output. Training uses Adam, gradient clipping, validation-loss checkpointing, learning-rate reduction, and early stopping. The trained model is then evaluated and used for stochastic text continuation with temperature, top-k, and top-p sampling. The report also covers data provenance, TensorFlow input semantics, numerical considerations, diagnostic metrics, artifact management, reproducibility controls, and implementation limitations. It introduces no unsupported experimental claims or result figures.

Contents

Configuration at a Glance

Parameter	Value
Seed	42
Sequence length	128 characters
Batch size	128
Embedding dimension	256
LSTM units per layer	512
Number of LSTM layers	2
Spatial dropout	0.15
Post-LSTM dropout	0.25
Initial learning rate	2×10^{-3}
Requested epochs	10
Training ratio	0.90
Gradient clip norm	1.0
Final generation length	1,000 new characters
Final temperature	0.72
Final top-k	20
Final top-p	0.88

1 Introduction

The analyzed program implements a character-level recurrent language model for Persian Shahnameh text. Its core learning task is next-character prediction: given a fixed-length character context, the model predicts a probability distribution over the characters in the learned vocabulary. The implementation uses a learned embedding layer followed by two stacked long short-term memory (LSTM) layers, a dense projection with GELU activation, and a softmax output layer.

The source is organized as a Google Colab-oriented experiment. It contains an original HTTP/Ganjoor acquisition subsystem, but the active execution path is redirected to a user-provided local file named `shahnameh.txt`. This distinction is important when establishing data provenance: the current pipeline trains from the local text file, while the remote acquisition helpers remain in the source for fidelity to the supplied program.

The report describes the implementation as it exists rather than replacing it with an idealized language-model pipeline. Particular attention is given to the data boundary, character normalization, deterministic splitting, sequence construction, LSTM architecture, mixed-precision behavior, generation controls, evaluation metrics, output artifacts, and reproducibility mechanisms. The report intentionally does not reproduce experimental plots or numerical results, even though the program can generate them as part of its output bundle.

2 Project Overview

Conceptually, the executable path can be divided into seven stages. The program first configures the runtime and dependencies, then loads the local Shahnameh text and records provenance. It normalizes Persian characters, constructs record-like units from the source lines, and performs a deterministic 90/10 split at the record level. The training and validation texts are then encoded at character granularity and converted into TensorFlow datasets containing one-step-shifted input-target pairs.

The model receives sequences of 128 integer character IDs. Each ID is mapped to a 256-dimensional embedding, processed by two 512-unit LSTM layers, projected through a 256-unit GELU dense layer, and

converted into a vocabulary-sized probability vector with a softmax. Training uses Adam with an initial learning rate of 2×10^{-3} and gradient-norm clipping at 1.0. Model selection is based on validation loss, with learning-rate reduction and early stopping enabled.

After training, the best checkpoint is restored, the validation set is evaluated, and a 1,000-character continuation is sampled from the Persian seed phrase specified in the source code. Generation combines temperature scaling, top-k filtering, and nucleus (top-p) filtering. Additional diagnostics measure repeated character n-grams, n-gram novelty relative to the training text, and generated line-length statistics. The program finally serializes metrics, training history, provenance, metadata, and generated text into an output directory and packages the files into a ZIP archive.

- Input and provenance: local `shahnameh.txt` with SHA-256 recording.
- Normalization: Unicode normalization, Persian/Arabic character harmonization, and removal of non-Persian letter marks.
- Sequence modeling: character-level next-token prediction with a fixed context length of 128.
- Network: Embedding → LSTM → Dropout → LSTM → Dropout → Dense(GELU) → Dense(Softmax).
- Optimization: Adam, learning-rate plateau reduction, gradient clipping, early stopping, and best-checkpoint restoration.
- Generation: temperature + top-k + top-p sampling, with explicit exclusion of the padding token.
- Artifacts: dataset JSONL, split text files, vocabulary metadata, weights, history, metrics, plots, summary JSON, README, and ZIP bundle.

3 Technical Foundations

The implementation is a character-level language model. Let a normalized text corpus be represented as a sequence of characters $x_1, x_2, \ldots, x_T$. The learning objective is to estimate the conditional distribution of the next character given a context of up to 128 preceding characters. Unlike a word-level model, the vocabulary is derived directly from the unique characters appearing in the combined corpus.

The embedding layer converts a discrete character ID into a dense vector. If V is the vocabulary size and d is the embedding dimension, the embedding parameters form a matrix $E \in \mathbb{R}^{V \times d}$. For character index i, the corresponding embedding is the ith row of E. In this implementation, $d = 256$.

The recurrent core uses LSTM units. An LSTM maintains a hidden state and a cell state and controls information flow using input, forget, and output gates. For an input vector x_t and previous hidden state h_{t-1}, the standard formulation is given below. The implementation uses the standard Keras LSTM layer with 512 recurrent units in each of two stacked layers.

The final layer applies a softmax over the vocabulary. At time step t, the resulting vector assigns a probability to each candidate character. Training therefore corresponds to minimizing sparse categorical cross-entropy over the observed next-character targets.

LSTM gates

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \tag{1}$$
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \tag{2}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \tag{3}$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \tag{4}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{5}$$
$$h_t = o_t \odot \tanh(c_t). \tag{6}$$

Cross-entropy objective

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(y_i \mid x_i). \tag{7}$$

Softmax

$$p_\theta(k \mid x_{\le t}) = \frac{\exp(z_k)}{\sum_{j=1}^{V} \exp(z_j)}. \tag{8}$$

4 Data and Input Processing

The active loader is `load_local_shahnameh_dataset`, which expects `/content/shahnameh.txt` by default and is explicitly invoked with that path. If the file is absent, the function attempts to open the Google Colab upload dialog. If a file named `shahnameh.txt` is uploaded, that name is used; if exactly one file is uploaded under a different name, that file becomes the effective path. Outside Colab, failure to find the file produces a `FileNotFoundError`.

The file is read as UTF-8, with an optional UTF-8 byte-order mark removed by `read_text(encoding='utf-8-sig')`. The normalization function removes a BOM, canonicalizes line endings, applies Unicode NFC normalization, converts a set of Arabic-script variants to preferred Persian forms, removes left-to-right and right-to-left marks, and converts zero-width non-joiners to ordinary spaces. It then scans each line character by character, retaining spaces and Arabic-script letters in the U+0600-U+06FF block while discarding combining marks and other characters. Multiple consecutive spaces are collapsed and empty lines are removed.

This filtering step has a direct modeling consequence. It does not merely clean punctuation; it defines the effective alphabet. Characters outside the retained range are not represented in the vocabulary. Consequently, punctuation, Latin characters, digits, and symbols are absent from the normalized corpus when they fall outside the accepted character categories. The implementation therefore learns a deliberately restricted representation of the source text.

4 Data and Input Processing (continued)

After normalization, each non-empty source line becomes a `PoemRecord` with a synthetic identifier and title. The record fields are compatible with the original Ganjoor-derived record structure, but metre and formal couplet count are not available from the local file and are therefore stored as an empty string and one, respectively. If the source contains fewer than two records, the code switches to chunking the

normalized text into pieces of at least 5,000 characters (and never shorter than `SEQ_LENGTH + 1`) so that a train/validation split can still be formed.

The loader computes a SHA-256 checksum of the original local file and stores basic provenance, including raw character count, normalized character count, and number of records. The records are serialized as UTF-8 JSON Lines. The checksum is useful for identifying the exact input file used for a run, although it does not itself guarantee that every downstream computation is bit-for-bit identical across hardware and software environments.

Text normalization

$$\text{normalized text} = \text{filter}(\text{NFC}(\text{transliterate}(\text{raw text}))). \tag{9}$$

5 System Architecture and Code Organization

The source is organized as a single executable script with logically separated sections rather than a package with multiple modules. Configuration constants appear first, followed by dependency installation, runtime configuration, legacy acquisition helpers, the local loader, data-preparation utilities, model construction, generation utilities, callbacks, and the main experiment flow.

The code uses dataclasses to define `PoemRecord`, typed function signatures for most helpers, `pathlib.Path` for file management, NumPy for numerical conversion, pandas for metric serialization, and TensorFlow/Keras for dataset handling, model construction, optimization, training, and evaluation. The architecture is intentionally compact: data representation, preprocessing, learning, generation, evaluation, and artifact management all live in the same file.

Several components retained from the original remote-data version are not part of the active execution path. `fetch_json`, `full_url_to_api_url`, `collect_category_urls`, `fetch_category`, `collect_poem_paths`, and `fetch_poem` remain available, and the configuration still defines Ganjoor URLs. The current local pipeline does not invoke those network-facing helpers. This is consistent with the explicit source note that the local execution path is used while the original acquisition helpers are kept intact.

5 System Architecture and Code Organization (continued)

- Configuration constants: sequence length, batch size, model width, optimization parameters, generation parameters, paths, and retry settings.
- Data layer: `PoemRecord`, normalization, local loading, provenance, JSONL serialization, deterministic splitting, vocabulary construction, and TensorFlow dataset creation.
- Model layer: functional Keras graph with embedding, recurrent layers, projection, and softmax output.
- Generation layer: token filtering, context management, padding, probabilistic sampling, and text continuation.
- Diagnostics layer: repetition, novelty, line-length statistics, metric/history CSV files, and experiment summary JSON.
- Execution layer: callbacks, fit/evaluate calls, artifact persistence, ZIP packaging, and optional Colab download.

6 Detailed Methodology

The first computational stage is runtime setup. Dependencies are checked using `importlib.util.find_spec`; missing packages are installed with pip before TensorFlow is imported. Python hash seeding and TensorFlow deterministic-operation environment variables are configured before the TensorFlow import. Python's `random` module, NumPy, and TensorFlow's Keras random-seed utility are then initialized with `SEED = 42`.

When GPUs are detected, memory growth is enabled to avoid pre-allocating the entire GPU memory pool. Mixed precision is requested through the global `mixed_float16` policy, with a fallback to float32 if configuration fails. The final vocabulary projection is explicitly forced to float32. This is a cautious numerical choice because softmax probabilities and cross-entropy are sensitive to finite-precision behavior, even when internal computation uses lower precision.

The data split uses a dedicated `random.Random(SEED)` instance. The records are copied, shuffled, and partitioned into 90% training and 10% validation sets. If the split would leave validation empty, the last training record is moved into validation. Train and validation texts are formed by joining their record texts with newline separators. The implementation stores metadata about counts and character lengths to support later auditability.

Vocabulary construction is performed over `full_text = train_text + newline + val_text`. A `<PAD>` token is inserted at index zero and all other characters are sorted. The mapping is serialized in both directions. Using `full_text` rather than `train_text` alone means the model knows every character that appears in validation, even though it is not trained on the validation sequences. In a character model with a very small alphabet this distinction may be minor, but it remains a relevant implementation detail because it differs from a strict train-only vocabulary construction.

6 Detailed Methodology (continued)

The text is encoded into integer arrays. Sequence construction then partitions each encoded corpus into consecutive, non-overlapping blocks of length `seq_len + 1`. Each block is split into an input sequence and a target sequence shifted by one character. For example, a block $c_1, \ldots, c_{129}$ produces input $c_1, \ldots, c_{128}$ and target $c_2, \ldots, c_{129}$. Any final remainder shorter than 129 characters is discarded. The use of fixed non-overlapping blocks is a substantive design choice: the code does not construct all possible sliding windows of length 128.

Training batches are shuffled at the block level, repeated indefinitely, and prefetched. Training uses `drop_remainder=True`, so a final partial batch is discarded. The number of batches per epoch is computed using floor division by batch size. Validation is finite, not repeated, and uses ceiling division for validation steps; its final partial batch is retained. This creates a clear distinction between an indefinite training stream and a finite validation pass.

Next-character pairs

$$X = (c_1, \ldots, c_{128}), \qquad Y = (c_2, \ldots, c_{129}). \tag{10}$$

Number of full blocks

$$N_{\text{seq}} = \left\lfloor \frac{T}{L+1} \right\rfloor, \qquad L = 128. \tag{11}$$

7 Mathematical Formulation

Let V denote the vocabulary size, $L = 128$ the context length, $d = 256$ the embedding dimension, and $H = 512$ the number of units in each LSTM layer. For an input sequence of IDs $x_{1:L}$, the embedding layer produces $e_t = E[x_t]$, where E is a $V \times d$ parameter matrix. The first LSTM transforms the embedding sequence into hidden states $h_t^{(1)}$. Because `return_sequences=True`, the complete sequence of hidden states is passed to the second LSTM, which produces $h_t^{(2)}$ at every position.

Dropout is applied after each recurrent layer with a rate of 0.25, while `recurrent_dropout` is set to zero inside the LSTM layers. Before the first LSTM, `SpatialDropout1D` uses a rate of 0.15. The distinction matters: spatial dropout regularizes whole embedding feature channels across a time sequence, whereas the subsequent standard Dropout operates on the recurrent output tensor. The report describes these layers as implemented rather than assuming that they have the same effect.

The recurrent output at each time position is transformed by a 256-unit dense layer using GELU. The final dense layer maps the projection to V logits and applies softmax. Because the model returns a probability distribution at every time position, the loss can compare the predicted distribution at each position with the corresponding shifted target character.

7 Mathematical Formulation (continued)

The model is optimized with Adam at a learning rate of 0.002. Gradient clipping is configured by `clipnorm=1.0`, meaning the optimizer scales a gradient update when its norm exceeds the specified threshold. This is a practical stabilization mechanism for recurrent optimization, where exploding gradients can otherwise be problematic.

Embedding lookup

$$e_t = E[x_t], \qquad E \in \mathbb{R}^{V \times 256}. \tag{12}$$

Dense projection

$$z_t = \mathrm{GELU}(W_p h_t^{(2)} + b_p), \qquad z_t \in \mathbb{R}^{256}. \tag{13}$$

Vocabulary probabilities

$$p_t(k) = \frac{\exp(o_{t,k})}{\sum_{j=1}^{V} \exp(o_{t,j})}, \qquad k \in \{1, \ldots, V\}. \tag{14}$$

Perplexity

$$\mathrm{PPL} = \exp(\mathcal{L}). \tag{15}$$

8 Optimization and Learning Procedure

Training is orchestrated by `model.fit`. The requested maximum is 10 epochs, but the run can stop earlier because EarlyStopping monitors validation loss with patience four and restores the best weights. ReduceLROnPlateau monitors the same quantity and halves the learning rate after two epochs without improvement, subject to a minimum learning rate of 10^{-5}. ModelCheckpoint also monitors validation loss and stores only the best weight file.

The three controls act at different levels. Checkpointing controls which parameter state is persisted. Learning-rate reduction changes the optimizer step size when progress stalls. Early stopping terminates training when continued optimization does not improve validation loss within the configured patience window. Together, these mechanisms favor retaining the best observed validation state rather than the last epoch.

8 Optimization and Learning Procedure (continued)

After `fit` returns, the script explicitly reloads the checkpoint if it exists. This is slightly redundant with `restore_best_weights=True`, but it makes the post-training state explicit: the model used for final evaluation and generation is the saved best-weight state, assuming the checkpoint file was created successfully.

The per-epoch callback generates a short sample after every completed epoch using a fixed seed phrase and sampling configuration. Sampling errors are caught and logged rather than stopping training. This makes qualitative monitoring non-blocking, but it also means generation behavior is not a hard dependency for successful optimization.

9 Text Generation and Sampling

Generation starts by normalizing the seed text and removing any characters not found in the learned vocabulary. The initial sequence is then converted to integer IDs. For each of the 1,000 requested continuation steps, the last 128 token IDs are selected. If the current context is shorter than 128 characters, it is left-padded with the explicit PAD token.

The model predicts probabilities from the final time position of the context. The PAD probability is explicitly forced to zero before sampling. The remaining distribution is normalized and passed to `sample_token`. That function clips negative probabilities, renormalizes the vector, applies optional temperature scaling in log space, optionally keeps only the top-k values, and optionally applies nucleus filtering based on cumulative probability mass.

Temperature changes the sharpness of a probability distribution. For temperature τ, the implementation effectively transforms probabilities p_k into a normalized distribution proportional to $\exp(\log(p_k)/\tau)$. Values below one make the distribution sharper; values above one make it flatter. The program uses 0.72 in the global final-generation configuration and 0.75 for the epoch-end monitoring callback.

9 Text Generation and Sampling (continued)

Top-k retains the k highest-probability candidates and sets all others to zero. Top-p sorts probabilities from largest to smallest and removes candidates after the cumulative probability exceeds p, while always preserving the most probable candidate. In the main run, $k = 20$ and $p = 0.88$. The combination bounds the sampling support while retaining stochastic variation.

A subtle implementation detail is that `gen_length` counts newly appended characters, not total output length, because the seed text is inserted into `generated` before the loop and the loop appends `gen_length` additional characters. Therefore, the final string length is approximately `len(seed) + gen_length`, subject to the absence of skipped PAD samples.

Temperature transformation

$$q_k = \frac{\exp(\log p_k / \tau)}{\sum_j \exp(\log p_j / \tau)}. \tag{16}$$

Nucleus condition

$$\sum_{k \in S_p} p_k \geq p, \qquad p = 0.88. \tag{17}$$

10 Evaluation and Diagnostic Metrics

The script evaluates the restored model on the validation dataset and reports sparse categorical cross-entropy loss and character-level accuracy. Validation perplexity is computed as exp(loss), but the implementation caps the loss at 20 before exponentiation. This cap prevents numerical overflow in the diagnostic calculation; it also means the reported perplexity is not mathematically identical to exp(loss) when the loss exceeds 20.

The remaining metrics are diagnostics of the generated sequence rather than learned-model likelihood measures. The 4-gram repetition rate is defined as one minus the number of unique character 4-grams divided by the total number of 4-grams. A value of zero means every 4-gram is unique, while larger values indicate more repeated local patterns. The metric counts the seed portion as part of the generated string because it is computed on `final_generation` directly.

10 Evaluation and Diagnostic Metrics (continued)

The novelty metric compares generated 4-grams with the set of 4-grams observed in the training text. It returns the fraction of generated 4-gram occurrences whose exact character sequence was not observed in training. This is a surface-form novelty statistic, not a semantic novelty measure and not, by itself, an indicator of literary originality.

Line-length statistics count non-empty lines and then compute mean, standard deviation, and median character counts. Because the sampling loop does not explicitly generate line breaks according to a prosodic or metrical model, these values are useful only as descriptive diagnostics. They do not constitute a meter-scansion algorithm, a phonological analysis, or proof of formal poetic validity.

Repetition rate

$$R_n = 1 - \frac{|\,\mathrm{unique}(G_n)|}{|G_n|}. \tag{18}$$

Novelty

$$N_n = \frac{\sum_{g \in G_n} \mathbf{1}[g \notin T_n]}{|G_n|}, \tag{19}$$

where G_n is the list of generated n-grams and T_n is the set of training-text n-grams.

11 Implementation Details

The program uses `pathlib.Path` to construct paths under `/content` when that directory exists and otherwise falls back to the current working directory. The output directory contains distinct files for the dataset, cleaned corpus, vocabulary, model weights, generated text, training history, evaluation metrics, plots, experiment summary, README, and ZIP bundle.

11 Implementation Details (continued)

The dataset and provenance files are UTF-8 and preserve Persian characters by using `ensure_ascii=False`. The vocabulary JSON stores integer-to-character mappings with stringified keys because JSON object

keys must be strings. The source therefore preserves the semantic content of the reverse mapping while respecting JSON's representation constraints.

TensorFlow's data pipeline uses `deterministic=True` in the map transformation, a fixed shuffle seed, AUTOTUNE for parallel mapping and prefetching, and an explicit batch shape. These choices balance reproducibility with throughput. The code does not, however, guarantee perfect cross-platform determinism because TensorFlow kernels, GPU libraries, driver versions, and hardware can still influence floating-point execution.

The source includes several imports and configuration elements inherited from the original acquisition-oriented version. For example, `shutil`, `ThreadPoolExecutor`, and `as_completed` are not part of the active computational path visible in the supplied source, and the Ganjoor manifest/category/poet URL constants are not needed by the local loader. Their presence does not change current behavior, but it is relevant when assessing maintainability and minimality.

The plotting subsystem saves a PNG containing training and validation loss and accuracy curves. That file is deliberately excluded from this report to respect the no-figure requirement. The same applies to the generated text preview printed to the console: this report explains the mechanism without reproducing the generated output.

12 Execution Workflow

Execution begins with dependency availability checks and deterministic runtime configuration. The script then calls the local loader, producing `PoemRecord` instances and a provenance dictionary. The records are split into training and validation corpora, after which the joint vocabulary is built and both text partitions are integer encoded.

12 Execution Workflow (continued)

Two TensorFlow datasets are then built. The training dataset is shuffled, repeated, batched with the remainder dropped, and prefetched; the validation dataset is finite and keeps the final partial batch. The model is constructed with the configured vocabulary size and architecture, and three callbacks are installed: checkpointing, learning-rate reduction, and early stopping. A fourth callback provides per-epoch sampling.

The training call consumes exactly `steps_per_epoch` batches for each epoch. After training, the best weight file is reloaded if present. The model is evaluated on validation batches, the final continuation is generated, and the generated sequence is written to disk. Metrics are then assembled into a table-like list and saved as CSV, while Keras history is serialized separately.

Finally, the experiment summary combines assignment metadata, architecture settings, training metadata, dataset provenance, split statistics, evaluation metrics, and generation settings into a single JSON object. A README note documents the interpretation of diagnostic metrics, and all generated files except the ZIP itself are added to a compressed archive. In Colab, the script attempts to trigger the browser download of that archive.

13 Design Decisions and Rationale

The strongest design decision is the explicit preservation of the original downstream pipeline while replacing remote acquisition with local input. Instead of rewriting the rest of the experiment around a new corpus abstraction, the loader synthesizes `PoemRecord` objects so that the pre-existing split and serialization functions can continue to operate. This minimizes changes to downstream logic and keeps the local adaptation structurally faithful to the supplied program.

Character-level modeling is also consistent with the goal of generating Persian text without requiring a separate tokenizer or morphological analyzer. The resulting vocabulary is small compared with word-level language modeling, while the model can represent unseen word boundaries through character composition. The trade-off is that longer dependencies require more recurrent steps and the model must learn orthographic regularities at a fine granularity.

A 128-character context is implemented as a hard window during training and generation. The choice limits computational cost per example and gives the LSTM a bounded historical view. It also means that dependencies beyond that context cannot be modeled directly by a single training example, even though the recurrent state can still encode structure within each 128-character window.

13 Design Decisions and Rationale (continued)

The sampling strategy intentionally avoids deterministic argmax decoding. Temperature, top-k, and top-p allow the user to tune the balance between conservatism and diversity. The explicit removal of PAD from the sampling distribution is especially appropriate because PAD exists for sequence-shape management and is not a semantic character of the corpus.

14 Computational Considerations

Let N_{seq} be the number of full training sequences and B the batch size. Building the encoded corpus is $O(T)$ in the number of characters T. The sequence-construction step is also $O(T)$, aside from TensorFlow dataset bookkeeping. Each training batch performs recurrent computation over 128 time steps and two LSTM layers with 512 units, followed by the dense projection and vocabulary projection.

For an LSTM with input width D and hidden width H, the dominant parameter count per recurrent layer is approximately $4H(D + H + 1)$, because four gated affine transformations are learned. In the first LSTM, $D = 256$; in the second, $D = 512$ because it consumes the first layer's hidden sequence. The final vocabulary projection adds parameters proportional to $256V$ plus bias, so vocabulary size directly affects the cost of producing the full softmax distribution.

14 Computational Considerations (continued)

The training dataset is intentionally bounded at the sequence level and is streamed through `tf.data`. Prefetching overlaps input preparation with model execution. Mixed precision can reduce memory traffic and accelerate compatible GPU kernels, but actual gains depend on hardware and TensorFlow support.

The code does not provide a custom memory budget, gradient accumulation mechanism, or distributed training strategy.

Generation is computationally sequential: each new character requires a model call conditioned on the latest context. Consequently, generating 1,000 characters involves approximately 1,000 forward passes, although each pass uses only one sequence of length 128. This differs fundamentally from teacher-forced training, where many target positions are processed in parallel for each batch.

LSTM parameter count

$$P_{\text{LSTM}} \approx 4H(D + H + 1). \tag{20}$$

Generation complexity

$$T_{\text{gen}} = O(G \cdot C_{128}), \tag{21}$$

where G is the number of generated characters and C_{128} denotes one 128-step forward pass through the model.

15 Reproducibility and Reliability

Several mechanisms explicitly support reproducibility. The global seed is fixed at 42; Python, NumPy, and TensorFlow random generators are initialized; TensorFlow deterministic-operation flags are set before import; the record split uses a seeded Random instance; the training dataset shuffle has an explicit seed; and dataset mapping is marked deterministic. The local input file is fingerprinted using SHA-256, while split and training metadata are saved alongside the model artifacts.

15 Reproducibility and Reliability (continued)

The experiment summary records the requested epoch count, the number of completed epochs, GPU names, and the active mixed-precision policy. Generation parameters are also stored, including temperature, top-k, top-p, seed prompt, requested length, and actual output length. This is stronger than relying solely on console output because the configuration is persisted as structured metadata.

Reproducibility is nevertheless conditional. TensorFlow's deterministic flags reduce sources of nondeterminism but cannot guarantee identical floating-point trajectories across all accelerator architectures, library versions, and software environments. The script also installs dependencies using unconstrained specifications such as `numpy` and `pandas`, while TensorFlow is bounded only by a lower version. Thus, the code captures the experimental configuration well but does not provide a fully pinned environment lockfile.

16 Limitations and Technical Considerations

The local adaptation changes the semantic meaning of the original `PoemRecord` abstraction. Each source line becomes a synthetic record, and the fields `metre` and `couplets_count` are placeholders rather than observed metadata from a structured poetic corpus. This is a practical compatibility layer, not a reconstruction of the original Ganjoor record semantics.

The train/validation split is performed before character sequence construction, which helps prevent direct mixing of record-level content across the split. However, when the local source contains very short lines, each line is treated as a separate record. The resulting validation set may therefore differ substantially from a poem-level split. The source does not perform higher-level grouping or document-level holdout beyond the line-based compatibility representation.

16 Limitations and Technical Considerations (continued)

Vocabulary construction uses both training and validation text. Again, this does not train on validation sequences, but it exposes the character inventory of validation to the model. For a strict experimental protocol, building the vocabulary from the training corpus alone would provide a cleaner separation. That change, however, would be a methodological modification rather than a faithful description of the current code.

The sequence generator discards remainder characters that do not fit an integer number of 129-character blocks. This simplifies tensor shapes and batch accounting, but some corpus characters are therefore absent from the training objective. The training batch calculation can discard an additional partial batch because `drop_remainder=True`.

The text normalization procedure removes combining marks and retains Arabic-script letters only. This makes the dataset consistent and reduces alphabet variation, but it can also remove meaningful orthographic or punctuation information. The program intentionally favors a normalized character inventory over exact reproduction of every source symbol.

Finally, the program is primarily a language-modeling implementation rather than a Persian poetry modeling system in the formal linguistic sense. The generated line-length and n-gram diagnostics do not model metre, rhyme, syntax, semantics, or literary quality. The source itself documents this limitation by warning that these statistics are diagnostic proxies rather than proof of perfect meter.

17 Overall Technical Assessment

The implementation is technically coherent and follows a complete machine-learning workflow from data ingestion to reproducible artifact packaging. Its strongest qualities are explicit configuration, clear separation of helper responsibilities, deterministic split logic, structured provenance, a conventional and well-defined LSTM architecture, practical training controls, and a generation routine that exposes several sampling controls without introducing a second model or tokenizer layer.

17 Overall Technical Assessment (continued)

From a software-engineering perspective, the use of dataclasses, typed signatures, pathlib, structured JSON metadata, CSV exports, checkpointing, and a final archive gives the single-file script a useful level of operational discipline. The local-file adaptation is particularly careful because it preserves the original record-oriented downstream interfaces instead of rewriting the entire pipeline.

The main technical limitations are similarly clear: the retained remote-acquisition code increases surface area, the active local loader uses synthetic record semantics, the vocabulary incorporates validation characters, the sequence builder uses non-overlapping blocks, and reproducibility is strong but not fully environment-pinned. None of these observations invalidate the implementation; they define its exact methodological scope.

For an academic portfolio, the project demonstrates practical understanding of recurrent sequence modeling, text normalization, data engineering, TensorFlow input pipelines, optimization control, probabilistic decoding, evaluation design, and reproducibility. The most defensible description is therefore a compact, research-oriented character-level language-modeling pipeline with a deliberate local-data adaptation and explicit experimental bookkeeping.

18 Conclusion

The supplied program implements a complete character-level Shahnameh language-modeling experiment around a two-layer LSTM. The active pipeline reads a local UTF-8 text file, normalizes the Persian character stream, creates a deterministic train/validation partition, constructs a character vocabulary with explicit padding, and prepares fixed-length next-character prediction sequences.

The model combines a 256-dimensional embedding, two 512-unit LSTM layers, dropout-based regularization, a GELU projection, and a vocabulary-sized softmax. Adam optimization is supplemented by gradient clipping, validation-loss checkpointing, learning-rate reduction, and early stopping. The final model supports stochastic generation through temperature, top-k, and top-p filtering, while the experiment records model, data, generation, and provenance metadata for later inspection.

18 Conclusion (continued)

Viewed as a software artifact, the project is notable less for algorithmic novelty than for the integration of modeling, preprocessing, runtime control, diagnostic metrics, and reproducibility into one executable workflow. Its limitations are identifiable directly from the implementation, which makes the project suitable for technical documentation and academic portfolio presentation without requiring claims beyond what the code can substantiate.

Source-to-Report Traceability

The report is grounded in the supplied source file. The principal traceability anchors are the configuration block (lines 33–70), dependency and runtime setup (76–128), normalization and local loading (239–400), deterministic corpus split and vocabulary/data construction (408–505), model construction (510–565), sampling and generation (570–648), diagnostic metrics (651–683), callback logic (689–703), and the executable experiment flow (708–957). These locations identify the implementation regions from which the technical explanations in this report were derived.

The source also contains a plot-generation block (lines 851–872). It is part of the program's output behavior but is intentionally not reproduced in this report because the report requirements prohibit figures, plots, and visual result displays.

The source contains a README-generation block that labels the dataset as a Ganjoor public export. The active local loader, however, records provenance as a user-provided `shahnameh.txt` file. This report follows the loader's actual data path when describing the experiment and treats the retained Ganjoor text as legacy compatibility material rather than the active input source.